

OSGi Working Group
OSGi Specification repo for
`org.osgi.service.repository`

Release 1.1.1-SNAPSHOT
September 2026



Copyright © 2000, 2026 Eclipse Foundation

LICENSE

Eclipse Foundation Specification License – v1.0

By using and/or copying this document, or the Eclipse Foundation document from which this statement is linked, you (the licensee) agree that you have read, understood, and will comply with the following terms and conditions:

Permission to copy, and distribute the contents of this document, or the Eclipse Foundation document from which this statement is linked, in any medium for any purpose and without fee or royalty is hereby granted, provided that you include the following on ALL copies of the document, or portions thereof, that you use:

- link or URL to the original Eclipse Foundation document.
- All existing copyright notices, or if one does not exist, a notice (hypertext is preferred, but a textual representation is permitted) of the form: "Copyright © [date-of-document] Eclipse Foundation, Inc. <<url to this license>>".

Inclusion of the full text of this NOTICE must be provided. We request that authorship attribution be provided in any software, documents, or other items or products that you create pursuant to the implementation of the contents of this document, or any portion thereof.

No right to create modifications or derivatives of Eclipse Foundation documents is granted pursuant to this license, except anyone may prepare and distribute derivative works and portions of this document in software that implements the specification, in supporting materials accompanying such software, and in documentation of such software, PROVIDED that all such works include the notice below. HOWEVER, the publication of derivative works of this document for use as a technical specification is expressly prohibited.

The notice is:

"Copyright © [date-of-document] Eclipse Foundation. This software or document includes material copied from or derived from [title and URI of the Eclipse Foundation specification document]."

Disclaimers

THIS DOCUMENT IS PROVIDED "AS IS," AND THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION MAKE NO REPRESENTATIONS OR WARRANTIES, EXPRESS OR IMPLIED, INCLUDING, BUT NOT LIMITED TO, WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, OR TITLE; THAT THE CONTENTS OF THE DOCUMENT ARE SUITABLE FOR ANY PURPOSE; NOR THAT THE IMPLEMENTATION OF SUCH CONTENTS WILL NOT INFRINGE ANY THIRD PARTY PATENTS, COPYRIGHTS, TRADEMARKS OR OTHER RIGHTS.

THE COPYRIGHT HOLDERS AND THE ECLIPSE FOUNDATION WILL NOT BE LIABLE FOR ANY DIRECT, INDIRECT, SPECIAL OR CONSEQUENTIAL DAMAGES ARISING OUT OF ANY USE OF THE DOCUMENT OR THE PERFORMANCE OR IMPLEMENTATION OF THE CONTENTS THEREOF.

The name and trademarks of the copyright holders or the Eclipse Foundation may NOT be used in advertising or publicity pertaining to this document or its contents without specific, written prior permission. Title to copyright in this document will at all times remain with copyright holders.

Preface

Implementation Requirements

An implementation of a Specification: (i) must fully implement the Specification including all its required interfaces and functionality; (ii) must not modify, subset, superset or otherwise extend the OSGi Name Space, or include any public or protected packages, classes, Java interfaces, fields or methods within the OSGi Name Space other than those required and authorized by the Specification. An implementation that does not satisfy limitations (i) (ii) is not considered an implementation of the Specification and must not be described as an implementation of the Specification. "OSGi Name Space" shall mean the public class or interface declarations whose names begin with "org.osgi" or any recognized successors or replacements thereof. An implementation of a Specification must not claim to be a compatible implementation of the Specification unless it passes the Technology Compatibility Kit ("TCK") for the Specification.

Feedback

This specification can be downloaded from the OSGi Documentation web site:

<https://docs.osgi.org/specification/>

Comments about this specification can be raised at:

<https://github.com/osgi/osgi/issues>

Table of Contents

1	Introduction	5
1.1	Reader Level	5
1.2	Version Information	5
1.3	References	5
132	Repository Service Specification	7
132.1	Introduction	7
132.2	Using a Repository	8
132.3	Repository	12
132.4	osgi.content Namespace	12
132.5	XML Repository Format	13
132.6	XML Repository Schema	17
132.7	Capabilities	20
132.8	Security	21
132.9	org.osgi.service.repository	21
132.10	References	27

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

1 Introduction

This document contains a specification developed by the OSGi working group.

1.1 Reader Level

This specification is written for the following audiences:

- Application developers
- Framework and system service developers (system developers)
- Architects

This specification assumes that the reader has at least one year of practical experience in writing Java programs. Experience with embedded systems and server-environments is a plus. Application developers must be aware that the OSGi environment is significantly more dynamic than traditional desktop or server environments.

System developers require a very deep understanding of Java. At least three years of Java coding experience in a system environment is recommended. A Framework implementation will use areas of Java that are not normally encountered in traditional applications. Detailed understanding is required of class loaders, garbage collection, Java 2 security, and Java native library loading.

Architects should focus on the introduction of each subject. This introduction contains a general overview of the subject, the requirements that influenced its design, and a short description of its operation as well as the entities that are used. The introductory sections require knowledge of Java concepts like classes and interfaces, but should not require coding experience.

Most of these specifications are equally applicable to application developers and system developers.

1.2 Version Information

Each OSGi specification has an independent version, and packages defined by those specifications have their own versions as well. Specification and package versions follow semantic versioning rules.

1.3 References

- [1] *OSGi Specifications*
<https://docs.osgi.org/specification/>

DRAFT

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

132 Repository Service Specification

Version 1.1

132.1 Introduction

The guiding force behind the OSGi Specifications is a reusable component model. The ??? provides a solid foundation for such a component model by providing a component collaboration framework with a comprehensive management model. The service specifications provide the abstract APIs to allow many different collaborations between components. This Repository Service Specification provides the capability to manage the external access to components and other resources.

Though the Repository service can be used as a standalone service to search and retrieve general binary artifacts, called resources, it is intended to be used in conjunction with the [6] *Resolver Service Specification*.

The model of the Repository is based on the generic Requirement-Capability model defined in [3] *Resource API Specification*, this chapter relies on the definitions of the generic model.

132.1.1 Essentials

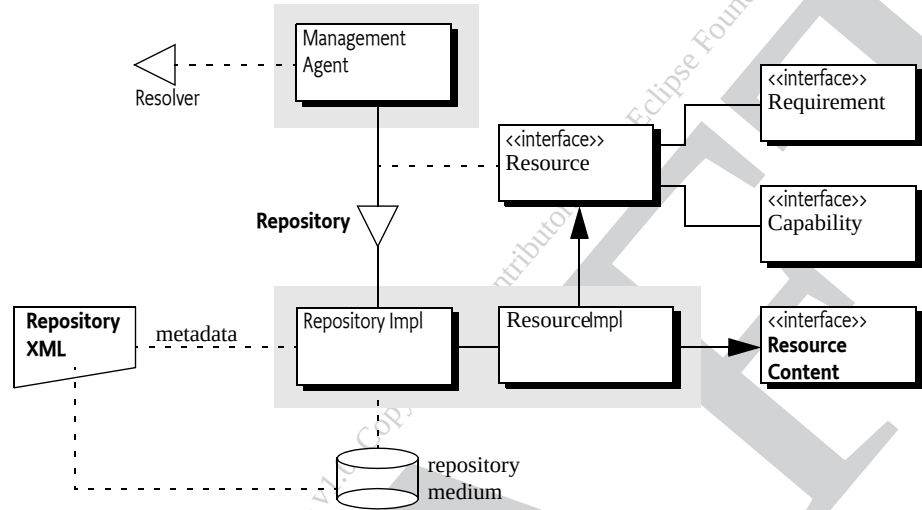
- *External* - Provide access to external components and resources.
- *Resolve* - The Repository API must be closely aligned with the Resolver API since they are intended to be used in conjunction.
- *Searching* - Support general queries.
- *Metadata* - Allow resources to provide content information.
- *Retrieval* - Allow the retrieval of Resources from remote locations.
- *Batching* - Repositories must be able to batch queries.
- *Distribution* - Allow Repositories to be defined with a simple storage scheme such that Repositories can be distributed on a removable media like a CD/DVD.
- *Mirroring* - Repositories must be able to support selecting a remote site based on the local situation.

132.1.2 Entities

- *Repository* - A facade to a (remote) set of resources described by capabilities.
- *Resource* - An artifact that has requirements that must be satisfied before it is available but provides capabilities when it becomes available.
- *Requirement* - An expression that asserts a capability.
- *Capability* - Describes a feature of the resource so that it can be required by a requirement.
- *Resource Content* - Provides access to the underlying bytes of the resource in the default format.

Figure 132.1

Class and Service overview



132.1.3

Synopsis

There are many different repositories available on the Internet or on fixed media. A repository can be made available to bundles by providing a Repository service. If such a bundle, for example a Management Agent performing a provisioning operation, finds that it has an unmatched requirement then it can query the repository services to find matching capabilities. The Repository service can implement the query in many different ways. It can ship the requirement to a remote side to be processed or it can process the query locally.

This specification also provides an XML schema that can be used to describe a Repository. Instances of this schema can be downloaded from a remote repository for local indexing or they can be stored for example on a DVD together with the resources.

132.2

Using a Repository

The Repository service provides an abstraction to a, potentially remote, set of resources. In the generic Capability-Requirement model, resources are modeled to declare capabilities and requirements. The primary purpose of a Repository is to enable a management agent that uses the Resolver API to leverage a wide array of repositories. This Repository service specification allows different Repository providers to be installed as bundles, and each bundle can register multiple Repository services. The Repository is sufficiently abstract to allow many different implementations.

Repository services are identified by a number of service properties:

- `service.pid` - A mandatory unique identity for this Repository service.
- `service.description` - An optional human readable name for this Repository.
- `repository.url` - Optional URLs to landing pages of the repository, if they exist.

In general, the users of the Repository service should aggregate all services in the service registry. This strategy allows the deployer to control the available Repositories. The following example, using Declarative Service annotations to show the dependencies on the service registry, shows how to aggregate the different Repository services.

```
List<Repository> repos = new CopyOnWriteArrayList<Repository>();
```

```

@Reference(
cardinality = ReferenceCardinality.MULTIPLE,
policy = ReferencePolicy.DYNAMIC)
void addRepository( Repository repo ) { repos.add(repo); }
void removeRepository( Repository repo ) { repos.remove(repo); }

```

To access a resource in a Repository service it is necessary to construct a requirement, pass this to the Repository service, and then use the returned capabilities to satisfy the resolver or to get the resource from the capability. The Repository then returns all matching capabilities. The requirement matches the capability if their namespaces match and the requirement's filter is absent or matches the attributes.

The [findProviders\(Collection\)](#) method takes a Collection of requirements. The reason for this collection is that it allows the caller to specify multiple requirements simultaneously so that Repositories can batch requests, the requirements in this collection are further unrelated. That is, they do not form an expression in any way. Multiple requirements as the parameter means that the result must be a map so that the caller can find out what requirement matched what capabilities. For example:

```

List<Capability> find( Requirement r ){
    List<Capability> result = new ArrayList<Capability>();

    for ( Repository repo : repos ) {
        Map<Requirement,Collection<Capability>> answer =
            repo.findProviders( Collections.singleton( r ) );
        result.addAll( answer.get( r ) );
    }
    return result;
}

```

Access to resources is indirect since the Repository returns capabilities. Each capability is declared in a resource and the `getResource()` method provides access to the underlying resource. Since each resource declares an `osgi.identity` capability it is possible to retrieve a resource from a repository if the identity name, type, and version are known. For example, to get a bundle resource:

```

Resource getResource( String type, String name, Version version ) {
    String filter = String.format(
        "(%s=%s)(osgi.identity=%s)(version=%s)",
        type,
        name,
        version );

    RequirementBuilder builder = repo.newRequirementBuilder("osgi.identity");
    builder.addDirective("filter", filter);
    Requirement r = builder.build();

    List<Capability> capabilities = find( r );
    if ( capabilities.isEmpty() )
        return null;
    return capabilities.get( 0 ).getResource();
}

```

Resources that originate from Repository services must implement the [RepositoryContent](#) interface, this interface provides stream access to the default storage format. It is therefore possible to get the content with the following code.

```

InputStream getContent( String type, String name, Version version ) {

```

```

Resource r = getResource( type, name, version );
if ( r == null )
    return null;
return ((RepositoryContent)r).getContent();
}

```

The `getContent()` method returns an Input Stream in the default format for that resource type. Resources from a Repository should also have one or more `osgi.content` capabilities that advertise the same resource in the same or different formats. The `osgi.content` capability has a number of attributes that provide information about the resource's download format:

- `osgi.content` - A unique SHA-256 for the content as read from the URL.
- `url` - A URL to the content.
- `mime` - An IANA MIME type for the content.
- `size` - Size in bytes of the content.

It is therefore possible to search for a specific MIME type and download that format. For example:

```

String getURL( String type, String name, Version version, String mime )
    throws Exception {
    Resource r = getResource( type, name, version );
    for ( Capability cap : r.getCapabilities( "osgi.content" ) ) {
        Map<String, Object> attrs = cap.getAttributes();
        String actual = (String) attrs.get( "mime" );
        if ( actual != null && mime.equalsIgnoreCase( actual ) ) {
            String url = (String) attrs.get( "url" );
            if ( url != null )
                return url;
        }
    }
    return null;
}

```

Since the `osgi.content` capability contains the SHA-256 digest as the `osgi.content` attribute it is possible to verify the download that it was correct.

Every resource has an `osgi.identity` capability. This namespace defines, in [2] *Framework Namespaces*, the possibility to add related resources, for example *javadoc* or *sources*. A resource then has informational requirements to `osgi.identity` capabilities; these requirements are marked with a classifier directive that holds the type of *relation*. The following example shows how it would be possible to find such a related resource:

```

InputStream getRelated( Resource resource, String classifier )
    throws Exception {
    for ( Requirement r : resource.getRequirements( "osgi.identity" ) ) {
        if ( classifier.equals( r.getDirectives().get( "classifier" ) ) ) {
            Collection<Capability> capabilities =
                repository.findProviders( Collections.singleton( r ) ).get( r );

            if ( capabilities.isEmpty() )
                continue;

            Capability c = capabilities.iterator().next();
            Resource related = c.getResource();
            return ((RepositoryContent)related).getContent();
        }
    }
}

```

```

    }
    return null;
}

```

132.2.1 Combining Requirements

In some cases it may be useful to find resources in the repository that satisfy criteria across multiple namespaces.

A simple Requirement object can contain a filter that makes assertions about capability attributes within a single namespace. So for example, a single requirement can state that a package org.example.mypkg must be exported in a version between 3.1 inclusive and 4.0 exclusive:

```

RequirementBuilder rb = repo.newRequirementBuilder("osgi.wiring.package");
String rf = "(&(osgi.wiring.package=org.example.mypkg)"
    + "(version>=3.1) (! (version>=4.0)))";
rb.addDirective("filter", rf);
Requirement r = rb.build();

```

This requirement contains three conditions on the osgi.wiring.package capability.

In some situations it may be needed to specify requirements that cover multiple namespaces. For example a bundle might be needed that exports the above package, but the bundle must also have the Apache License, Version 2.0 license. A resource's license is available as an attribute on the osgi.identity namespace. Constructing a constraint that combines requirements from multiple namespaces can be done by using an Expression Combiner, which can be obtained from the Repository service. The Repository service provides a [findProviders\(RequirementExpression\)](#) overload that can take a requirement expression and returns a Promise to a collection of matching resources.

```

RequirementBuilder lb = repo.newRequirementBuilder("osgi.identity");
String lf = "(license=http://opensource.org/licenses/Apache-2.0)";
lb.addDirective("filter", lf);

RequirementExpression expr = repo.getExpressionCombiner().and(
    lb.buildExpression(), rb.buildExpression());

Promise<Collection<Resource>> p = repo.findProviders(expr);

// Let findProviders() do its work async and update a ui component
// once the result is available
p.then(new Success<Collection<Resource>, Void>() {
    public Promise<Void> call(Promise<Collection<Resource>> resolved)
        throws Exception {
        ui.update(resolved.getValue());
        return null;
    }
});

// Instead of the async chain above its also possible to
// wait for the promise value synchronously:
// Collection<Resource> resources = p.getValue();

```

For more details on OSGi Promises, see the ???.

132.3 Repository

A Repository service provides access to capabilities that satisfy a given requirement. A Repository can be the facade of a remote server containing a large amount of resources, a repository on removable media, or even a collection of bundles inside a ZIP file. A Repository communicates in terms of requirements and capabilities as defined in [3] *Resource API Specification*. This model is closely aligned with the [6] *Resolver Service Specification*.

A Repository service must be registered with the service properties given in the following table.

Table 132.1

Repository Service Properties

Attribute	Opt	Type	Description
service.pid	mandatory	String	A globally unique identifier for this Repository.
service.description	optional	String	The Repository Name
repository.url	optional	String+	URLs related to this Repository.

The Repository implements the following methods:

- [findProviders\(Collection\)](#) - For each requirement find all the capabilities that match that requirement and return them as a `Map<Requirement, Collection<Capability>>`.
- [findProviders\(RequirementExpression\)](#) - Find all resources that match the requirement expression. The requirement expression is used to combine multiple requirements using the and, or and not operators.
- [getExpressionCombiner\(\)](#) - Obtain an expression combiner. This expression combiner is used to produce requirement expressions from simple requirements or other requirement expressions.
- [newRequirementBuilder\(String\)](#) - Obtain a convenience builder for Requirement objects.

A Repository must not perform any namespace specific actions or matching. The Repository must therefore match a requirement to a capability with the following rules:

- The namespace must be identical, and
- The requirement's filter is absent or it must match the capability's attributes.

Resources originating from a Repository service must additionally:

- Implement the [RepositoryContent](#) interfaces, see *Repository Content* on page 12.
- Provide at least one `osgi.content` Capability, see *osgi.content Namespace* on page 12.

132.3.1

Repository Content

Resources originating from a Repository must implement the [RepositoryContent](#) interface. The purpose of this interface is to allow users of the Repositories access to an Input Stream that provides access to the resource.

The [RepositoryContent](#) interface provides a single method:

- [getContent\(\)](#) - Return an Input Stream for the resource, if more than one `osgi.content` capability is present the content associated with the first capability is returned.

132.4 osgi.content Namespace

A resource is a logical concept, to install a resource in an environment it is necessary to get access to its *contents*. A resource can be formatted in different ways. It is possible to deliver a bundle as a JAR file, a Pack200 file, or some other format. In general, the [RepositoryContent](#) interface provides access to the default format.

The Repository can advertise the different formats with `osgi.content` capabilities. Each of those capabilities is identified with a unique SHA-256 checksum and has a URL for the resource in the specified format. The size and mime attributes provide information the download format, this can be used for selection. If more than one `osgi.content` capability is associated with a resource, the first capability must represent the default format. If the resource has a standard or widely used format (e.g., JAR for bundles and ESA for subsystems), and that format is provided as part of the repository, then that format should be the default format.

The `osgi.content` Namespace supports the attributes defined in the following table and [Content-Namespaces](#).

Table 132.2 *osgi.content definition*

Name	Kind	M/O	Type	Syntax	Description
osgi.content	CA	M	String	[0-9a-fA-F]{64}	The SHA-256 hex encoded digest for this resource
url	CA	M	String	<url>	The URL to the bytes. This must be an absolute URL.
size	CA	M	Long	[0-9]+	The size of the resource in bytes as it will be read from the URL.
mime	CA	M	String	<mime type>	An IANA defined MIME type for the format of this content.

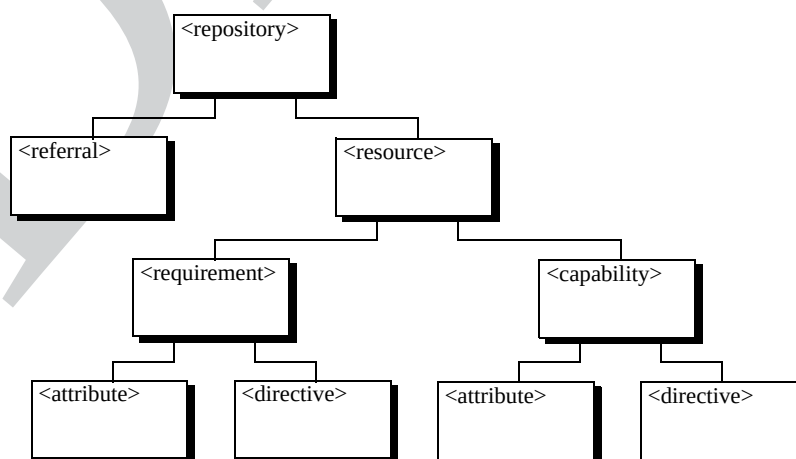
132.5 XML Repository Format

This is an optional part of the specification since the Repository interface does not provide access how the Repository obtains its information. However, the purpose of this part of the specification is to provide a commonly recognized format for interchanging Repository metadata.

This section therefore describes an XML schema to represent Repository content. It is expected that Internet based Repositories can provide such an XML file to clients. A Repository XML file can be used as a common interchange format between multiple Repository implementations.

The Repository XML describes a number of resources with their capabilities and requirements. Additionally the XML can refer to other Repository XML files. The XML Schema can be found at its XML namespace, see *XML Repository Schema* on page 17. The XML structure, which closely follows the Requirement-Capability model, is depicted in Figure 132.2.

Figure 132.2 *XML Structure*



The different elements are discussed in the following sections. All types are derived from the XML Schema types, see [4] *XML Schema Part 2: Data types Second Edition* . Any relative URIs in a Repository XML file must be resolved as specified in [5] *XML Base (Second Edition), Resolving Relative URIs* .

132.5.1 Repository Element

The repository element is the root of the document. The repository element has the following child elements:

- referral* - Referrals to other repositories for a federated model, see *Referral Element* on page 14 .
- resource* - Resource definitions, see *Resource Element* on page 14 .

The repository element has the attributes defined in the following table.

Table 132.3 repository element attributes

Attribute	Type	Description
name	NCName	The name of this Repository. For informational purposes.
increment	long	Counter which increments every time the repository is changed. Can be used by clients to check for changes. The counter is not required to increase monotonically.

132.5.2 Referral Element

The purpose of the referral element is to allow a Repository to refer to other Repositories, allowing for federated Repositories. Referrals are applied recursively. However, this is not always desired. It is therefore possible to limit the depth of referrals. If the depth attribute is ≥ 1 , the referred repository must be included but it must not follow any referrals from the referred repository. If the depth attribute is more than one, referrals must be included up to the given depth. Depths of referred repositories must also be obeyed, where referred repositories may reduce the effective depth but not increase it. For example if a top repository specifies a depth of 5 and a level 3 repository has a depth of 1 then the repository on level 5 must not be used. If not specified then there is no limit to the depth. Referrals that have cycles must be ignored, a resource of a given Repository must only occur once in a Repository.

The referral element has the attributes defined in the following table.

Table 132.4 referral element attributes

Attribute	Type	Description
depth	int	The max depth of referrals
url	anyURI	A URL to where the referred repository XML can be found. The URL can be absolute or relative to the URI of the current XML resource.

132.5.3 Resource Element

The resource element defines a Resource. The resource element has the following child elements:

- requirement* - The requirements of this resource, see *Requirement Element* on page 15 .
- capability* - The capabilities of this resource, see *Capability Element* on page 14 .

The Resource element has no attributes.

132.5.4 Capability Element

The capability element maps to a capability, it holds the attributes and directives. The capability element has the following child elements:

- **directive*** - The directives for the capability, see *Directive Element* on page 16 .
- **attribute*** - The attributes for the capability, see *Attribute Element* on page 15 .

The capability element has the attributes defined in the following table.

Table 132.5 *capability element attributes*

Attribute	Type	Description
namespace	token	The namespace of this capability

132.5.5 Requirement Element

The requirement element maps to a requirement, it holds the attributes and directives. The requirement element has the following child elements:

- **directive*** - The directives for the requirement, see *Directive Element* on page 16 .
- **attribute*** - The attributes for the requirement, see *Attribute Element* on page 15 .

The requirement element has the attributes defined in the following table.

Table 132.6 *requirement element attributes*

Attribute	Type	Description
namespace	token	The namespace of this requirement

132.5.6 Attribute Element

An attribute element describes an attribute of a capability or requirement. Attributes are used to convey information about the Capability-Requirement. Attributes for the capability are used for matching the requirement's filter. The meaning of attributes is described with the documentation of the namespace in which they reside.

Attributes are optionally typed according to the [1] *Framework Module Layer* specification. The default type is String, the value of the `value` attribute. However, if a type attribute is specified and it is not String then the value attribute must be converted according to the type attribute specifier. The syntax of the type attribute is as follows:

```
type    ::= list | scalar
list    ::= 'List<' scalar '>'           // no spaces between terminals
scalar  ::= 'String' | 'Version' | 'Long' | 'Double'
```

A list conversion requires the value to be broken in tokens separated by comma (',' \u002C). White-space around the list and around commas must be trimmed for non-String types. Each token must then be converted to the given type according to the scalar type specifier. The exact rules for the comma separated lists are defined in [1] *Framework Module Layer* , see *Bundle Capability Attributes*.

The conversion of value `s`, when scalar, must take place with the following methods:

- String - No conversion, use `s`
- Version - `Version.parseVersion(s)`
- Long - After trimming whitespace, `Long.parseLong(s)`
- Double - After trimming whitespace, `Double.parseDouble(s)`

The attribute element has the attributes defined in the following table.

Table 132.7 *attribute element attributes*

Attribute	Type	Description
name	token	The name of the attribute

Attribute	Type	Description
value	string	The value of the attribute.
type		The type of the attribute, the syntax is outlined in the previous paragraphs.

132.5.7 Directive Element

A directive element describes a directive of a capability or a requirement. Directives are used to convey information about the Capability-Requirement. The meaning of directives is described with the documentation of the namespace in which they reside.

The directive element has the attributes defined in the following table.

Table 132.8 directive element attributes

Attribute	Type	Description
name	token	The name of the attribute
value	string	The value of the attribute.

132.5.8 Sample XML File

The following example shows a very small XML file. The file contains one resource.

```
<repository name='OSGiRepository'
  increment='13582741'
  xmlns='http://www.osgi.org/xmlns/repository/v1.0.0'>
  <resource>

    <requirement namespace='org.osgi.wiring.package'>
      <directive name='filter' value=
        '(&org.osgi.wiring.package=org.apache.commons.pool)(version>=1.5.6)'/>
    </requirement>

    <requirement namespace='org.osgi.identity'>
      <directive name='effective' value='meta'/>
      <directive name='resolution' value='optional'/>
      <directive name='filter' value=
        '(&(version=1.5.6)(org.osgi.identity=org.acme.pool-src))'>
      <directive name='classifier' value='sources'/>
    </requirement>

    <capability namespace='org.osgi.identity'>
      <attribute name='org.osgi.identity' value='org.acme.pool'/>
      <attribute name='version' type='Version' value='1.5.6'/>
      <attribute name='type' value='org.osgi.bundle'/>
    </capability>

    <capability namespace='org.osgi.content'>
      <attribute name='org.osgi.content' value='e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855'>
      <attribute name='url' value='http://www.acme.com/repository/org/acme/pool/org.acme.pool-1.5.6.jar'/>
      <attribute name='size' type='Long' value='4405'/>
      <attribute name='mime' value='application/vnd.osgi.bundle'/>
    </capability>

    <capability namespace='org.osgi.wiring.bundle'>
      <attribute name='org.osgi.wiring.bundle' value='org.acme.pool'/>
      <attribute name='bundle-version' type='Version' value='1.5.6'/>
    </capability>

    <capability namespace='org.osgi.wiring.package'>
      <attribute name='org.osgi.wiring.package' value='org.acme.pool'/>
      <attribute name='version' type='Version' value='1.1.2'/>
      <attribute name='bundle-version' type='Version' value='1.5.6'/>
      <attribute name='bundle-symbolic-name' value='org.acme.pool'/>
      <directive name='uses' value='org.acme.pool,org.acme.util'/>
    </capability>

  </resource>
</repository>
```

132.6 XML Repository Schema

The namespace of this schema is:

`http://www.osgi.org/xmlns/repository/v1.0.0`

The schema for this namespace can be found at the location implied in its name. The recommended prefix for this namespace is `repo`.

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
  Copyright (c) Contributors to the Eclipse Foundation

  Licensed under the Apache License, Version 2.0 (the "License");
  you may not use this file except in compliance with the License.
  You may obtain a copy of the License at

    http://www.apache.org/licenses/LICENSE-2.0

  Unless required by applicable law or agreed to in writing, software
  distributed under the License is distributed on an "AS IS" BASIS,
  WITHOUT WARRANTIES OR CONDITIONS OF ANY KIND, either express or implied.
  See the License for the specific language governing permissions and
  limitations under the License.

  SPDX-License-Identifier: Apache-2.0
-->
<schema xmlns="http://www.w3.org/2001/XMLSchema"
  xmlns:repo="http://www.osgi.org/xmlns/repository/v1.0.0"
  targetNamespace="http://www.osgi.org/xmlns/repository/v1.0.0"
  elementFormDefault="unqualified"
  attributeFormDefault="unqualified"
  version="1.0.1">

  <element name="repository" type="repo:Trepository" />
  <complexType name="Trepository">
    <sequence>
      <choice minOccurs="0" maxOccurs="unbounded">
        <element name="resource" type="repo:Tresource" />
        <element name="referral" type="repo:Treferral" />
      </choice>
      <!-- It is non-deterministic, per W3C XML Schema 1.0:
      http://www.w3.org/TR/xmlschema-1/#cos-nonambig
      to use name space="##any" below. -->
      <any namespace="##other" processContents="lax" minOccurs="0"
        maxOccurs="unbounded" />
    </sequence>
    <attribute name="name" type="string">
      <annotation>
        <documentation xml:lang="en">
          The name of the repository. The name may contain
          spaces and punctuation.
        </documentation>
      </annotation>
    </attribute>
    <attribute name="increment" type="long">
      <annotation>
        <documentation xml:lang="en">
          An indication of when the repository was last changed. Client's can
          check if a
          repository has been updated by checking this increment value.
        </documentation>
      </annotation>
    </attribute>
    <anyAttribute processContents="lax" />
  </complexType>

  <complexType name="Tresource">
    <annotation>
      <documentation xml:lang="en">
        Describes a general resource with
```

```

        requirements and capabilities.
    </documentation>
</annotation>
<sequence>
    <element name="requirement" type="repo:Trequirement" minOccurs="0" maxOccurs="unbounded"/>
    <element name="capability" type="repo:Tcapability" minOccurs="1" maxOccurs="unbounded"/>
    <!-- It is non-deterministic, per W3C XML Schema 1.0:
    http://www.w3.org/TR/xmlschema-1/#cos-nonambig
    to use name space="##any" below. -->
    <any namespace="##other" processContents="lax" minOccurs="0"
        maxOccurs="unbounded" />
</sequence>
<anyAttribute processContents="lax" />
</complexType>

<complexType name="Treferral">
    <annotation>
        <documentation xml:lang="en">
            A referral points to another repository XML file. The
            purpose of this element is to create a federation of
            repositories that can be accessed as a single
            repository.
        </documentation>
    </annotation>
    <attribute name="depth" type="int" use="optional">
        <annotation>
            <documentation xml:lang="en">
                The depth of referrals this repository acknowledges.
            </documentation>
        </annotation>
    </attribute>
    <attribute name="url" type="anyURI" use="required">
        <annotation>
            <documentation xml:lang="en">
                The URL to the referred repository. The URL can be
                absolute or relative from the given repository's
                URL.
            </documentation>
        </annotation>
    </attribute>
    <anyAttribute processContents="lax" />
</complexType>

<complexType name="Tcapability">
    <annotation>
        <documentation xml:lang="en">
            A named set of type attributes and directives. A capability can be
            used to resolve a requirement if the resource is included.
        </documentation>
    </annotation>
    <sequence>
        <choice minOccurs="0" maxOccurs="unbounded">
            <element name="directive" type="repo:Tdirective" />
            <element name="attribute" type="repo:Tattribute" />
        </choice>
        <!-- It is non-deterministic, per W3C XML Schema 1.0:
        http://www.w3.org/TR/xmlschema-1/#cos-nonambig
        to use name space="##any" below. -->
        <any namespace="##other" processContents="lax" minOccurs="0"
            maxOccurs="unbounded" />
    </sequence>
    <attribute name="namespace" type="string">
        <annotation>
            <documentation xml:lang="en">
                Name space of the capability. Only requirements with the
                same name space must be able to match this capability.
            </documentation>
        </annotation>
    </attribute>
    <anyAttribute processContents="lax" />
</complexType>

<complexType name="Trequirement">
    <annotation>
        <documentation xml:lang="en">

```

```

    A filter on a named set of capability attributes.
  </documentation>
</annotation>
<sequence>
  <choice minOccurs="0" maxOccurs="unbounded">
    <element name="directive" type="repo:Tdirective" />
    <element name="attribute" type="repo:Tattribute" />
  </choice>
  <!-- It is non-deterministic, per W3C XML Schema 1.0:
  http://www.w3.org/TR/xmlschema-1/#cos-nonambig
  to use name space="##any" below. -->
  <any namespace="##other" processContents="lax" minOccurs="0"
    maxOccurs="unbounded" />
</sequence>
<attribute name="namespace" type="string">
  <annotation>
    <documentation xml:lang="en">
      Name space of the requirement. Only capabilities within the
      same name space must be able to match this requirement.
    </documentation>
  </annotation>
</attribute>
<anyAttribute processContents="lax" />
</complexType>

<complexType name="Tattribute">
  <annotation>
    <documentation xml:lang="en">
      A named value with an optional type that decorates
      a requirement or capability.
    </documentation>
  </annotation>
  <sequence>
    <any namespace="##any" processContents="lax" minOccurs="0"
      maxOccurs="unbounded" />
  </sequence>
  <attribute name="name" type="string">
    <annotation>
      <documentation xml:lang="en">
        The name of the attribute.
      </documentation>
    </annotation>
  </attribute>
  <attribute name="value" type="string">
    <annotation>
      <documentation xml:lang="en">
        The value of the attribute.
      </documentation>
    </annotation>
  </attribute>
  <attribute name="type" type="repo:TpropertyType" default="String">
    <annotation>
      <documentation xml:lang="en">
        The type of the attribute.
      </documentation>
    </annotation>
  </attribute>
  <anyAttribute processContents="lax" />
</complexType>

<complexType name="Tdirective">
  <annotation>
    <documentation xml:lang="en">
      A named value of type string that instructs a resolver
      how to process a requirement or capability.
    </documentation>
  </annotation>
  <sequence>
    <any namespace="##any" processContents="lax" minOccurs="0"
      maxOccurs="unbounded" />
  </sequence>
  <attribute name="name" type="string">
    <annotation>
      <documentation xml:lang="en">
        The name of the directive.

```

```

        </documentation>
      </annotation>
    </attribute>
    <attribute name="value" type="string">
      <annotation>
        <documentation xml:lang="en">
          The value of the directive.
        </documentation>
      </annotation>
    </attribute>
    <anyAttribute processContents="lax" />
  </complexType>

  <simpleType name="TpropertyType">
    <restriction base="string">
      <enumeration value="String" />
      <enumeration value="Version" />
      <enumeration value="Long" />
      <enumeration value="Double" />
      <enumeration value="List<String>" />
      <enumeration value="List<Version>" />
      <enumeration value="List<Long>" />
      <enumeration value="List<Double>" />
    </restriction>
  </simpleType>
  <attribute name="must-understand" type="boolean" default="false">
    <annotation>
      <documentation xml:lang="en">
        This attribute should be used by extensions to documents to require that
        the document consumer understand the extension. This attribute must be
        qualified when used.
      </documentation>
    </annotation>
  </attribute>
</schema>

```

132.7 Capabilities

Implementations of the Repository Service specification must provide the capabilities listed in this section.

132.7.1 osgi.implementation Capability

The Repository Service implementation bundle must provide the `osgi.implementation` capability with name `osgi.repository`. This capability can be used by provisioning tools and during resolution to ensure that a Repository Service implementation is present. The capability must also declare a `uses` constraint for the `org.osgi.service.repository` package and provide the version of this specification:

```

Provide-Capability: osgi.implementation;
                   osgi.implementation="osgi.repository";
                   uses:="org.osgi.service.repository";
                   version:Version="
1.1
"
```

This capability must follow the rules defined for the ???.

132.7.2 osgi.service Capability

The Repository Service implementation must provide a capability in the `osgi.service` namespace representing the [Repository](#) service. This capability must also declare a `uses` constraint for the `org.osgi.service.repository` package. For example:

```

Provide-Capability: osgi.service;
```

```
objectClass:List<String>="org.osgi.service.repository.Repository";
uses:="org.osgi.service.repository"
```

This capability must follow the rules defined for the ???.

132.8 Security

132.8.1 External Access

Repositories in general will get their metadata and artifacts from an external source, which makes them an attack vector for a malevolent Bundle that needs unauthorized external access. Since a Bundle using a Repository has no knowledge of what sources the Repository will access it will be necessary for the Repository to implement the external access in a `doPrivileged` block. Implementations must ensure that callers cannot influence/modify the metadata in such a way that the `getContent()` method could provide access to arbitrary Internet resources. This could for example happen if:

- The implementation relies on the `org.osgi.content` namespace to hold the URL
- The attributes Map from the `org.osgi.content` Capability is modifiable

If the malevolent Bundle could change the `org.osgi.content` attribute it could change it to arbitrary URLs. This example should make it clear that Repository implementations must be very careful.

132.8.2 Permissions

Implementations of this specification will need the following minimum permissions.

```
ServicePermission[...Repository, REGISTER ]
SocketPermission[ ... carefully restrict external access...]
```

Users of this specification will need the following minimum permissions.

```
ServicePermission[...Repository, GET ]
```

132.9 org.osgi.service.repository

Repository Service Package Version 1.1.

Bundles wishing to use this package must list the package in the Import-Package header of the bundle's manifest. This package has two types of users: the consumers that use the API in this package and the providers that implement the API in this package.

Example import for consumers using the API in this package:

```
{@code Import-Package: org.osgi.service.repository; version="[1.1,2.0)"}
```

Example import for providers implementing the API in this package:

```
{@code Import-Package: org.osgi.service.repository; version="[1.1,1.2)"}
```

132.9.1 Summary

- `AndExpression` - A `{@link RequirementExpression}` representing the `{@code and}` of a number of requirement expressions.
- `ContentNamespace` - Content Capability and Requirement Namespace.
- `ExpressionCombiner` - An `{@code ExpressionCombiner}` can be used to combine requirement expressions into a single complex requirement expression using the `{@code and}`, `{@code or}` and `{@code not}` operators.

- `IdentityExpression` - A {@link RequirementExpression} representing a requirement.
- `NotExpression` - A {@link RequirementExpression} representing the {@code not} (negation) of a requirement expression.
- `OrExpression` - A {@link RequirementExpression} representing the {@code or} of a number of requirement expressions.
- `Repository` - A repository service that contains {@link Resource} resources.
- `RepositoryContent` - An accessor for the content of a resource.
- `RequirementBuilder` - A builder for requirements.
- `RequirementExpression` - The super interface for all requirement expressions.

132.9.2 **public abstract AndExpression extends RequirementExpression**

A {@link RequirementExpression} representing the {@code and} of a number of requirement expressions.

Since @since 1.1

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

132.9.2.1 **public abstract List<RequirementExpression> getRequirementExpressions()**

- Return the requirement expressions that are combined by this {@code AndExpression}.

Returns An unmodifiable list of requirement expressions that are combined by this {@code AndExpression}. The list contains the requirement expressions in the order they were specified when this requirement expression was created.

132.9.3 **public final class ContentNamespace extends Namespace**

Content Capability and Requirement Namespace.

This class defines the names for the attributes and directives for this namespace. All unspecified capability attributes are of type {@code String} and are used as arbitrary matching attributes for the capability. The values associated with the specified directive and attribute keys are of type {@code String}, unless otherwise indicated.

Concurrency Immutable

132.9.3.1 **public static final String CAPABILITY_MIME_ATTRIBUTE = "mime"**

The capability attribute that defines the IANA MIME Type/Format for this content.

132.9.3.2 **public static final String CAPABILITY_SIZE_ATTRIBUTE = "size"**

The capability attribute that contains the size, in bytes, of the content. The value of this attribute must be of type {@code Long}.

132.9.3.3 **public static final String CAPABILITY_URL_ATTRIBUTE = "url"**

The capability attribute that contains the URL to the content.

132.9.3.4 **public static final String CONTENT_NAMESPACE = "osgi.content"**

Namespace name for content capabilities and requirements.

Also, the capability attribute used to specify the unique identifier of the content. This identifier is the {@code SHA-256} hash of the content.

132.9.4 public abstract ExpressionCombiner

An {@code ExpressionCombiner} can be used to combine requirement expressions into a single complex requirement expression using the {@code and}, {@code or} and {@code not} operators.

Since @since 1.1

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

132.9.4.1 public abstract AndExpression and(RequirementExpression expr1, RequirementExpression expr2)

expr1 The first requirement expression to combine into the returned requirement expression.

expr2 The second requirement expression to combine into the returned requirement expression

- Combine two {@link RequirementExpression}s into a requirement expression using the {@code and} operator.

Returns An {@link AndExpression} representing an {@code and} of the specified requirement expressions.

132.9.4.2 public abstract AndExpression and(RequirementExpression expr1, RequirementExpression expr2, RequirementExpression[]... moreExprs)

expr1 The first requirement expression to combine into the returned requirement expression.

expr2 The second requirement expression to combine into the returned requirement expression

moreExprs Optional, additional requirement expressions to combine into the returned requirement expression.

- Combine multiple {@link RequirementExpression}s into a requirement expression using the {@code and} operator.

Returns An {@link AndExpression} representing an {@code and} of the specified requirement expressions.

132.9.4.3 public abstract IdentityExpression identity(Requirement req)

req The requirement to wrap in a requirement expression.

- Wrap a {@link Requirement} in an {@link IdentityExpression}. This can be useful when working with a combination of {@code Requirement}s and {@code RequirementExpression}s.

Returns An {@link IdentityExpression} representing the specified requirement.

132.9.4.4 public abstract NotExpression not(RequirementExpression expr)

expr The requirement expression to negate.

- Return the negation of a {@link RequirementExpression}.

Returns A {@link NotExpression} representing the {@code not} of the specified requirement expression.

132.9.4.5 public abstract OrExpression or(RequirementExpression expr1, RequirementExpression expr2)

expr1 The first requirement expression to combine into the returned requirement expression.

expr2 The second requirement expression to combine into the returned requirement expression

- Combine two {@link RequirementExpression}s into a requirement expression using the {@code or} operator.

Returns An {@link OrExpression} representing an {@code or} of the specified requirement expressions.

132.9.4.6 public abstract OrExpression or(RequirementExpression expr1, RequirementExpression expr2, RequirementExpression[]... moreExprs)

expr1 The first requirement expression to combine into the returned requirement expression.

expr2 The second requirement expression to combine into the returned requirement expression

moreExprs Optional, additional requirement expressions to combine into the returned requirement expression.

- Combine multiple {@link RequirementExpression}s into a requirement expression using the {@code or} operator.

Returns An {@link OrExpression} representing an {@code or} of the specified requirement expressions.

132.9.5 **public abstract IdentityExpression extends RequirementExpression**

A {@link RequirementExpression} representing a requirement.

Since @since 1.1

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

132.9.5.1 **public abstract Requirement getRequirement()**

- Return the {@link Requirement} contained in this {@code IdentityExpression}.

Returns The requirement contained in this {@code IdentityExpression}.

132.9.6 **public abstract NotExpression extends RequirementExpression**

A {@link RequirementExpression} representing the {@code not} (negation) of a requirement expression.

Since @since 1.1

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

132.9.6.1 **public abstract RequirementExpression getRequirementExpression()**

- Return the requirement expression that is negated by this {@code NotExpression}.

Returns The requirement expression that is negated by this {@code NotExpression}.

132.9.7 **public abstract OrExpression extends RequirementExpression**

A {@link RequirementExpression} representing the {@code or} of a number of requirement expressions.

Since @since 1.1

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

132.9.7.1 **public abstract List<RequirementExpression> getRequirementExpressions()**

- Return the requirement expressions that are combined by this {@code OrExpression}.

Returns An unmodifiable list of requirement expressions that are combined by this {@code OrExpression}. The list contains the requirement expressions in the order they were specified when this requirement expression was created.

132.9.8 **public abstract Repository**

A repository service that contains {@link Resource} resources.

Repositories may be registered as services and may be used as by a resolve context during resolver operations.

Repositories registered as services may be filtered using standard service properties.

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

132.9.8.1 **public static final String URL = "repository.url"**

Service property to provide URLs related to this repository.

The value of this property must be of type {@code String}, {@code String[]}, or {@code Collection<String>}.

132.9.8.2 **public abstract Map<Requirement, Collection<Capability>> findProviders(Collection<? extends Requirement> requirements)**

requirements The requirements for which matching capabilities should be returned. Must not be {@code null}.

- Find the capabilities that match the specified requirements.

Returns A map of matching capabilities for the specified requirements. Each specified requirement must appear as a key in the map. If there are no matching capabilities for a specified requirement, then the value in the map for the specified requirement must be an empty collection. The returned map is the property of the caller and can be modified by the caller. The returned map may be lazily populated, so calling {@code size()} may result in a long running operation.

132.9.8.3 **public abstract Promise<Collection<Resource>> findProviders(RequirementExpression expression)**

expression The {@code RequirementExpression} for which matching capabilities should be returned. Must not be {@code null}.

- Find the resources that match the specified requirement expression.

Returns A promise to a collection of matching {@code Resource}s. If there are no matching resources, an empty collection is returned. The returned collection is the property of the caller and can be modified by the caller. The returned collection may be lazily populated, so calling {@code size()} may result in a long running operation.

Since @since 1.1

132.9.8.4 **public abstract ExpressionCombiner getExpressionCombiner()**

- Return an expression combiner. An expression combiner can be used to combine multiple requirement expressions into more complex requirement expressions using {@link AndExpression and}, {@link OrExpression or} and {@link NotExpression not} operators.

Returns An {@code ExpressionCombiner}.

Since @since 1.1

132.9.8.5 **public abstract RequirementBuilder newRequirementBuilder(String namespace)**

namespace The namespace for the requirement to be created.

- Return a new {@code RequirementBuilder} which provides a convenient way to create a requirement.

For example:

```
Requirement myReq = repository.newRequirementBuilder("org.foo.ns1").
    addDirective("filter", "(org.foo.ns1=val1)").
    addDirective("cardinality", "multiple").build();
```

Returns A new requirement builder for a requirement in the specified namespace.

Since @since 1.1

132.9.9 **public abstract RepositoryContent**

An accessor for the content of a resource. All {@link Resource} objects which represent resources in a {@link Repository} must implement this interface. A user of the resource can then cast the {@link Resource} object to this type and then obtain an {@code InputStream} to the content of the resource.

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

132.9.9.1 **public abstract InputStream getContent()**

- Returns a new input stream to the content of this resource. The content is represented on the resource through the {@code osgi.content} capability. If more than one such capability is associated with the resource, the first such capability is returned.

Returns A new input stream for associated content.

132.9.10 **public abstract RequirementBuilder**

A builder for requirements.

Since @since 1.1

Provider Type Consumers of this API must not implement this type

132.9.10.1 **public abstract RequirementBuilder addAttribute(String name, Object value)**

name The attribute name.

value The attribute value.

- Add an attribute to the set of attributes.

Returns This requirement builder.

132.9.10.2 **public abstract RequirementBuilder addDirective(String name, String value)**

name The directive name.

value The directive value.

- Add a directive to the set of directives.

Returns This requirement builder.

132.9.10.3 **public abstract Requirement build()**

- Create a requirement based upon the values set in this requirement builder.

Returns A requirement created based upon the values set in this requirement builder.

132.9.10.4 **public abstract IdentityExpression buildExpression()**

- Create a requirement expression for a requirement based upon the values set in this requirement builder.

Returns A requirement expression created for a requirement based upon the values set in this requirement builder.

132.9.10.5 **public abstract RequirementBuilder setAttributes(Map<String, Object> attributes)**

attributes The map of attributes.

- Replace all attributes with the attributes in the specified map.

Returns This requirement builder.

132.9.10.6 **public abstract RequirementBuilder setDirectives(Map<String, String> directives)**

directives The map of directives.

- Replace all directives with the directives in the specified map.

Returns This requirement builder.

132.9.10.7 **public abstract RequirementBuilder setResource(Resource resource)**

resource The resource.

- Set the {@code Resource}.

A resource is optional. This method will replace any previously set resource.

Returns This requirement builder.

132.9.11 **public abstract RequirementExpression**

The super interface for all requirement expressions. All requirement expressions must extend this interface.

Since @since 1.1

Concurrency Thread-safe

Provider Type Consumers of this API must not implement this type

132.10 References

- [1] *Framework Module Layer*
OSGi Core, Chapter 3 Module Layer
- [2] *Framework Namespaces*
OSGi Core, Chapter 8, osgi.identity Namespace
- [3] *Resource API Specification*
OSGi Core, Chapter 6 Resource API Specification
- [4] *XML Schema Part 2: Data types Second Edition*
<https://www.w3.org/TR/xmlschema-2/>
- [5] *XML Base (Second Edition), Resolving Relative URIs*
<https://www.w3.org/TR/xmlbase/#resolution>
- [6] *Resolver Service Specification*
OSGi Core, Chapter 58 Resolver Service Specification

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT

End Of Document

Licensed under the Eclipse Foundation Specification License – v1.0. Copyright © Contributors to the Eclipse Foundation

DRAFT